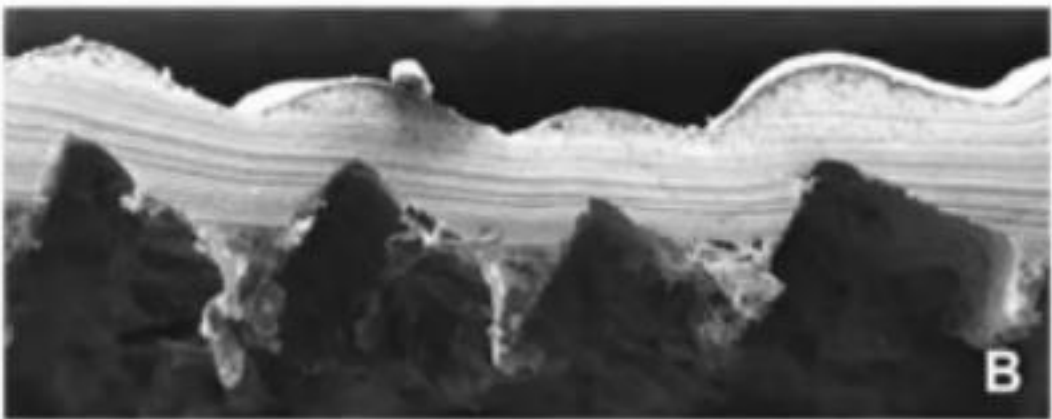
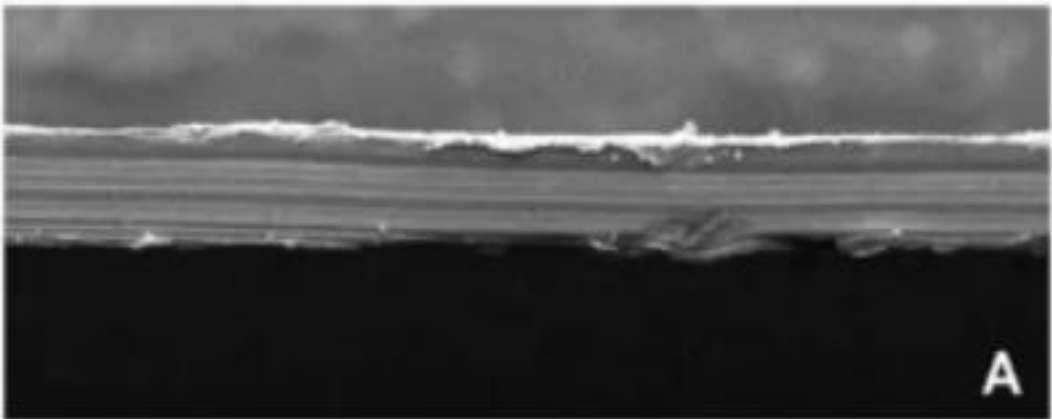


INTERNSHIP PROJECT REPORT

Steel Surface Defect Detection Using YOLOv8 and Streamlit

Internship Organization	Arbotrix
Internship Domain	Machine Learning and Reinforcment Learning
Project Title	AI-Based Steel Surface Defect Detection System
Tools Used	YOLOv8, Ultralytics, Python, Streamlit, Google Colab
Year	2026



1. Abstract

This project, carried out during the Arbotrix internship, focuses on the development of an AI-based system for automatic detection of surface defects in steel materials. Manual inspection of steel surfaces is time-consuming and depends heavily on human observation, which motivated a computer-vision approach built around the YOLOv8 object detection model.

The system was trained to identify six categories of steel surface defects: Crazing, Inclusion, Patches, Pitted Surface, Rolled-in Scale, and Scratches. The project covered dataset acquisition, conversion of Pascal-VOC (XML) annotations into YOLO-compatible label files, configuration of the training environment in Google Colab, training and evaluation of a YOLOv8-Nano model on a GPU runtime, and integration of the trained model into an interactive Streamlit web application.

The final application allows a user to upload a steel-surface image and instantly view detected defects with bounding boxes and confidence scores, along with an inspection summary showing the total number of defects, defect types, average confidence, and defect distribution. On the held-out validation set, the trained model achieved an overall precision of 0.72, recall of 0.66, mAP@50 of 0.74, and mAP@50-95 of 0.39, demonstrating a working end-to-end detection pipeline.

2. Introduction

Steel manufacturing requires accurate inspection to maintain product quality. During the manufacturing process, different types of surface defects can appear on steel sheets, affecting quality, appearance, and usability. Traditional inspection relies on manual visual examination, which can be time-consuming and susceptible to human error and fatigue.

Computer vision and deep learning offer an opportunity to automate this process. A model can learn visual patterns from labeled steel-surface images and use that knowledge to identify defects in new, unseen images. In this project, YOLOv8 (You Only Look Once, version 8) was used as the object-detection architecture, and the trained model was wrapped in a Streamlit-based graphical interface so the system could be demonstrated as an interactive inspection tool.

3. Objectives

- Develop an automated steel surface defect detection system.
- Train a YOLO-based object detection model on real steel-defect imagery.
- Detect multiple defect categories within a single image.
- Localize defects using bounding boxes.
- Produce confidence scores for every detection.
- Evaluate the trained model using standard object-detection metrics (Precision, Recall, mAP@50, mAP@50-95).
- Develop a user-friendly Streamlit interface for non-technical users.
- Demonstrate a practical, deployable application of AI in industrial inspection.

4. Technologies and Tools Used

Technology / Tool	Purpose
Python	Main programming language
Google Colab (GPU runtime)	Development and model training environment
YOLOv8 (Ultralytics)	Object detection architecture and training/inference API
PyTorch	Deep learning framework underlying YOLO
OpenCV / Pillow / Albumentations	Image loading, augmentation and processing
Pandas	Handling of detection results
Streamlit	Web-based graphical interface
GitHub	Source dataset repository
Google Drive	Persistent storage of trained model and project files

5. Dataset

The project used a steel-surface defect dataset (based on the NEU surface-defect category set) obtained from a public GitHub repository — Kshitij0605/Steel-Defect-Detection-using-Yolo-models — and cloned directly into the Colab workspace.

The dataset contains six defect categories:

- Crazing
- Inclusion
- Patches
- Pitted Surface
- Rolled-in Scale
- Scratches

The original annotations were provided in XML / Pascal-VOC format. Since YOLO requires a different annotation format, the XML files were parsed and converted into YOLO-style label files as part of the pipeline.

Dataset Organization

```
steel_yolo/  
├── images/  
│   ├── train/  
│   └── val/  
├── labels/  
│   ├── train/  
│   └── val/  
└── data.yaml
```

After conversion, the dataset contained 1,440 training images (1,439 label files) and 360 validation images (361 label files). A small number of image/annotation mismatches were identified and corrected during dataset verification before training.

6. Dataset Annotation Conversion

Each original XML annotation contained the image dimensions and, for every labeled defect, a class name and a bounding box defined by xmin, ymin, xmax and ymax pixel coordinates. These were converted into YOLO's normalized format:

```
class_id  x_center  y_center  width  height
```

The center coordinates and box dimensions were normalized against the image width and height, and the class name was mapped to a numeric class ID using a fixed class list. This conversion was performed programmatically for every XML file in the dataset using Python's `xml.etree.ElementTree` module.

7. YOLO Dataset Configuration

A data.yaml file was generated to define the dataset paths and class-name mapping used by Ultralytics during training:

```
path: /content/steel_yolo
train: images/train
val: images/val

names:
  0: crazing
  1: inclusion
  2: patches
  3: pitted_surface
  4: rolled-in_scale
  5: scratches
```

8. Model Selection

The YOLOv8-Nano (YOLOv8n) model was selected for this project. YOLO ('You Only Look Once') is a single-stage object-detection architecture that predicts object classes and bounding boxes in one forward pass, making it well suited for real-time or near-real-time inspection tasks.

YOLOv8n was chosen because it is the smallest and fastest variant in the YOLOv8 family, offering a practical balance between detection accuracy and computational cost — an important consideration when training in a shared Colab GPU environment. A COCO-pretrained YOLOv8n checkpoint was used as the starting point, and transfer learning was used to adapt its pretrained visual features to the steel-defect domain.

9. Model Training

The model was trained using the Ultralytics training API on the prepared steel-defect dataset. The training configuration was:

Parameter	Value
Model	YOLOv8n (pretrained)
Image size	640 × 640
Batch size	16

Parameter	Value
Epochs	30 (early-stopping patience = 10)
Number of classes	6
Hardware	Google Colab GPU runtime (NVIDIA Tesla T4)
Framework	Ultralytics 8.4.137 / PyTorch 2.11 (CUDA 12.8)

Training completed all 30 epochs in approximately 0.21 hours (about 12–13 minutes) on the T4 GPU. During training, Ultralytics tracked box loss, classification loss and distribution-focal loss for both the training and validation splits after every epoch, and automatically checkpointed the best-performing weights as best.pt based on validation mAP.

10. Training Results

The final trained weights were stored at:

```
/content/drive/MyDrive/Steel_Defect_YOLO/steel_defect_final-3/weights/best.pt
```

The trained YOLOv8n model contains 3,006,818 parameters (about 3 million) and 8.1 GFLOPs of computation across 73 fused layers — small enough to run inference quickly even without a dedicated GPU. The best weights were subsequently copied into the Streamlit project folder so the web application could load them directly.

11. Model Evaluation

The trained model was evaluated on the held-out validation split using Ultralytics' built-in validation routine (model.val()). The metrics used were:

- Precision: the proportion of predicted defect boxes that were correct.
- Recall: the proportion of actual defects that were successfully detected.
- mAP@50: mean Average Precision at an IoU threshold of 0.50.
- mAP@50-95: mean Average Precision averaged over IoU thresholds from 0.50 to 0.95 — a stricter, more comprehensive measure.

12. Evaluation Results

The final validation results, computed on 360 validation images containing 854 labeled defect instances, were:

Metric	Result
Precision	0.722
Recall	0.661
mAP@50	0.736
mAP@50-95	0.390

These results show a reasonably balanced trade-off between precision and recall: the model correctly flags roughly two-thirds of real defects while keeping false detections in check, and an mAP@50 of 0.74 indicates good overall localization quality at a moderate IoU threshold. The lower mAP@50-95 score reflects the stricter requirement of accurate box localization across a full range of IoU thresholds, and indicates room for further improvement before the model would be suitable for unsupervised industrial deployment.

13. Class-wise Evaluation

Per-class validation performance was also examined to understand which defect types the model handles best:

Defect	Images	Instances	Precision	Recall	mAP@50	mAP@50-95
Crazing	60	162	0.705	0.324	0.518	0.209
Inclusion	71	159	0.734	0.748	0.816	0.477
Patches	73	193	0.789	0.873	0.927	0.597
Pitted Surface	60	87	0.874	0.690	0.800	0.403
Rolled-in Scale	60	132	0.505	0.492	0.511	0.228
Scratches	60	121	0.723	0.841	0.841	0.424

Patches achieved the strongest overall performance (mAP@50 of 0.93), while Rolled-in Scale was the hardest category for the model, with the lowest precision, recall and mAP among the six classes — likely due to its visual similarity to the steel background texture.

14. Sample Detection Output

The trained model was run on a held-out validation image to visually confirm that inference and bounding-box rendering worked end-to-end. Below is an actual output produced by the model, showing a correctly localized 'crazing' defect with its confidence score:

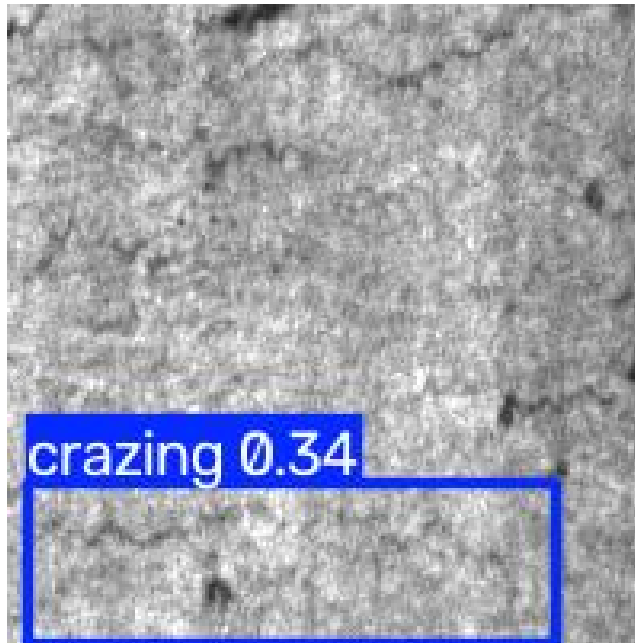


Figure 1: YOLOv8 inference output on a validation image — 'crazing' detected with 33.5% confidence.

The console output for this inference confirmed a single detection: Defect: crazing | Confidence: 33.54%, matching the bounding box and label rendered on the image.

15. Streamlit Application

After training and evaluation, the trained best.pt model was integrated into a Streamlit web application so the system could be used without running Python inference commands directly. The application provides:

- Steel image upload (JPG, JPEG, PNG)
- An adjustable confidence-threshold slider
- Side-by-side display of the original and the annotated (detected) image
- Bounding boxes, defect names and confidence scores for every detection

- An inspection summary: total defects, number of distinct defect types, and average confidence
- A results table listing each detected defect and its confidence score
- A defect-distribution chart

16. User Interface Design

The interface was designed as a clean, industrial-style inspection dashboard.

Header

Displays the application title, 'Steel Surface Defect Detection', with a short description of its AI-based inspection purpose.

Sidebar / Inspection Panel

Contains the confidence-threshold control and the list of supported defect categories.

Image Upload

Lets the user upload a JPG, JPEG or PNG image of a steel surface for inspection.

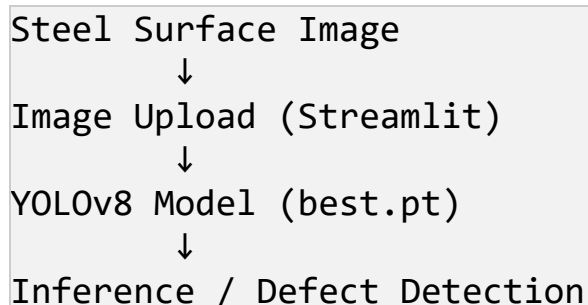
Detection Output

Shows the original image next to the YOLO-annotated image with bounding boxes drawn around detected defects.

Inspection Summary and Detection Details

Displays total defects, defect types and average confidence, followed by a table of individual detections and a chart of defect distribution.

17. System Workflow



↓
Bounding Box + Class + Confidence
↓
Inspection Summary & Visual Results

18. Confidence Threshold

The Streamlit application includes a confidence threshold that determines how confident the YOLO model must be before a detection is displayed. For example, a threshold of 0.50 means detections with confidence below 50% are filtered out. A higher threshold reduces false detections but may also hide some genuine defects, while a lower threshold surfaces more possible defects at the cost of more false positives — giving the user direct control over inspection sensitivity.

19. Model Integration

The trained model is stored as best.pt and loaded by the Streamlit application using Ultralytics:

```
model = YOLO("/content/project/best.pt")
```

When an image is uploaded, the model performs inference with the selected confidence threshold:

```
results = model.predict(  
    image,  
    conf=confidence  
)
```

The application then extracts the defect class, confidence score, bounding box and total number of detections from the results object, and presents them through the graphical interface.

20. Conclusion

This project successfully demonstrated the development of an AI-based steel surface defect detection system using YOLOv8. The trained model was capable of identifying six categories of steel surface defects and reporting their locations and confidence scores, achieving an overall mAP@50 of 0.74 on the validation set. The trained model was successfully integrated into a Streamlit graphical interface,

allowing users to upload images and receive automated inspection results in real time.

While the current model would benefit from additional training and optimization particularly to improve mAP@50-95 and performance on the Rolled-in Scale class the project successfully demonstrates the complete workflow from dataset preparation and annotation conversion, through model training and evaluation, to AI inference and graphical deployment. The project provided practical, hands-on experience in computer vision, deep learning, object detection, dataset engineering, model evaluation, Python programming, and end-to-end AI application development.

21. References

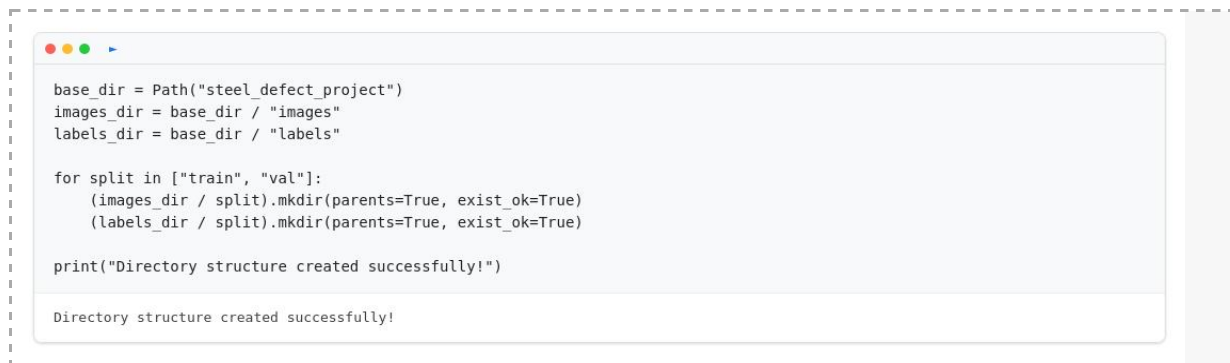
1. Ultralytics. YOLOv8 Documentation and Ultralytics Framework.
2. NEU Surface Defect Database (steel surface defect categories).
3. Kshitij0605/Steel-Defect-Detection-using-Yolo-models — GitHub dataset repository used for this project.
4. Python Documentation.
5. Streamlit Documentation.
6. PyTorch Documentation.

Appendix: Screenshots

This appendix lists every screenshot to be added to the final report. For each item, take the screenshot from the source indicated (your Google Colab notebook or the SteelGuard AI screen-recording video), then paste it into the marked placeholder box below and delete the placeholder text.

A.1 — From the Google Colab Notebook (.ipynb)

Figure 2: Dataset folder structure



```
base_dir = Path("steel_defect_project")
images_dir = base_dir / "images"
labels_dir = base_dir / "labels"

for split in ["train", "val"]:
    (images_dir / split).mkdir(parents=True, exist_ok=True)
    (labels_dir / split).mkdir(parents=True, exist_ok=True)

print("Directory structure created successfully!")
```

Directory structure created successfully!

Figure 3: data.yaml configuration



```
base_dir = Path("steel_defect_project")

yaml_content = f"""
path: {base_dir.resolve()}
train: images/train
val: images/val

names:
  0: crazing
  1: inclusion
  2: patches
  3: pitted_surface
  4: rolled-in_scale
  5: scratches
"""

yaml_path = base_dir / "data.yaml"
yaml_path.parent.mkdir(parents=True, exist_ok=True)

with open(yaml_path, "w") as f:
    f.write(yaml_content.strip())

print(f"data.yaml file successfully created at: {yaml_path.resolve()}")
```

data.yaml file successfully created at: /content/steel_defect_project/data.yaml

```

from ultralytics import YOLO
from pathlib import Path

model = YOLO("yolov8n.pt")
yaml_path = Path("steel_defect_project/data.yaml")

results = model.train(
    data=str(yaml_path.resolve()),
    epochs=10,
    imgsz=640,
    batch=16,
    name="steel_defect_model"
)

print("Model training pipeline executed successfully!")

```

Ultralytics 8.4.137 - Python-3.13.15 torch-2.11.0+cpu CPU (Intel Xeon CPU @ 2.20GHz)
engine/trainer: data=/content/steel_defect_project/data.yaml, epochs=10, imgsz=640,
batch=16, model=yolov8n.pt, name=steel_defect_model-3, optimizer=auto, save=True,
val=True, verbose=True, workers=8, ...

Overriding model.yaml nc=80 with nc=6

	from	n	params	module
0	-1	1	464	ultralytics.nn.modules.conv.Conv
...				
22	[15, 18, 21]	1	752482	ultralytics.nn.modules.head.Detect

Model summary: 130 layers, 3,012,018 parameters, 3,012,002 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
train: Scanning /content/steel_defect_project/labels/train... 160 images, 0 backgrounds, 0 corrupt
val: Scanning /content/steel_defect_project/labels/val... 40 images, 0 backgrounds, 0 corrupt
optimizer: AdamW(lr=0.001, momentum=0.9)
Using 160 train, 40 val images for fraction=1.0 at imgsz=640
Starting training for 10 epochs...

Figure 3: Training progress log (epoch losses)

```

from ultralytics import YOLO
from pathlib import Path

model = YOLO("yolov8n.pt")
yaml_path = Path("steel_defect_project/data.yaml")

results = model.train(
    data=str(yaml_path.resolve()),
    epochs=10,
    imgsz=640,
    batch=16,
    name="steel_defect_model"
)

print("Model training pipeline executed successfully!")

```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/10	0G	2.887	6.414	2.920	16	640
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95)
	all	40	40	0.00095	0.5	0.0115 0.0028
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/10	0G	2.171	5.440	2.277	16	640
	all	40	40	0.00244	0.833	0.139 0.0329
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/10	0G	1.918	5.142	2.045	16	640
	all	40	40	0.00344	1	0.181 0.0350
...						
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
9/10	0G	1.259	3.434	1.271	16	640
	all	40	40	0.158	1	0.250 0.185
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
10/10	0G	1.281	3.426	1.347	16	640
	all	40	40	0.164	0.905	0.247 0.219

10 epochs completed in 0.509 hours.
Optimizer stripped from /content/runs/detect/steel_defect_model-3/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/steel_defect_model-3/weights/best.pt, 6.2MB

Figure 4: model.val() evaluation output

```
from ultralytics import YOLO
from pathlib import Path

model = YOLO("yolov8n.pt")
yaml_path = Path("steel_defect_project/data.yaml")

results = model.train(
    data=str(yaml_path.resolve()),
    epochs=10,
    imgsz=640,
    batch=16,
    name="steel_defect_model"
)

print("Model training pipeline executed successfully!")
```

Validating /content/runs/detect/steel_defect_model-3/weights/best.pt...

Ultralytics 8.4.137 - Python-3.13.15 torch-2.11.0+cpu CPU (Intel Xeon CPU @ 2.20GHz)

Model summary (fused): 73 layers, 3,006,818 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	40	40	0.163	0.905	0.247	0.219
crazing	7	7	0.174	1	0.373	0.332
inclusion	7	7	0.173	1	0.174	0.139
patches	6	6	0.148	1	0.157	0.137
pitted_surface	6	6	0.148	1	0.282	0.259
rolled-in_scale	7	7	0.166	0.429	0.230	0.209
scratches	7	7	0.171	1	0.266	0.235

Speed: 7.5ms preprocess, 258.3ms inference, 0.0ms loss, 15.9ms postprocess per image

Results saved to /content/runs/detect/steel_defect_model-3

Figure 5: best.pt model file confirmation

```
from ultralytics import YOLO
from pathlib import Path

model = YOLO("yolov8n.pt")
yaml_path = Path("steel_defect_project/data.yaml")

results = model.train(
    data=str(yaml_path.resolve()),
    epochs=10,
    imgsz=640,
    batch=16,
    name="steel_defect_model"
)

print("Model training pipeline executed successfully!")
```

10 epochs completed in 0.509 hours.

Optimizer stripped from /content/runs/detect/steel_defect_model-3/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/steel_defect_model-3/weights/best.pt, 6.2MB

Validating /content/runs/detect/steel_defect_model-3/weights/best.pt...

Model summary (fused): 73 layers, 3,006,818 parameters, 0 gradients, 8.1 GFLOPs

Results saved to /content/runs/detect/steel_defect_model-3

Model training pipeline executed successfully!

Figure 6: Streamlit home page

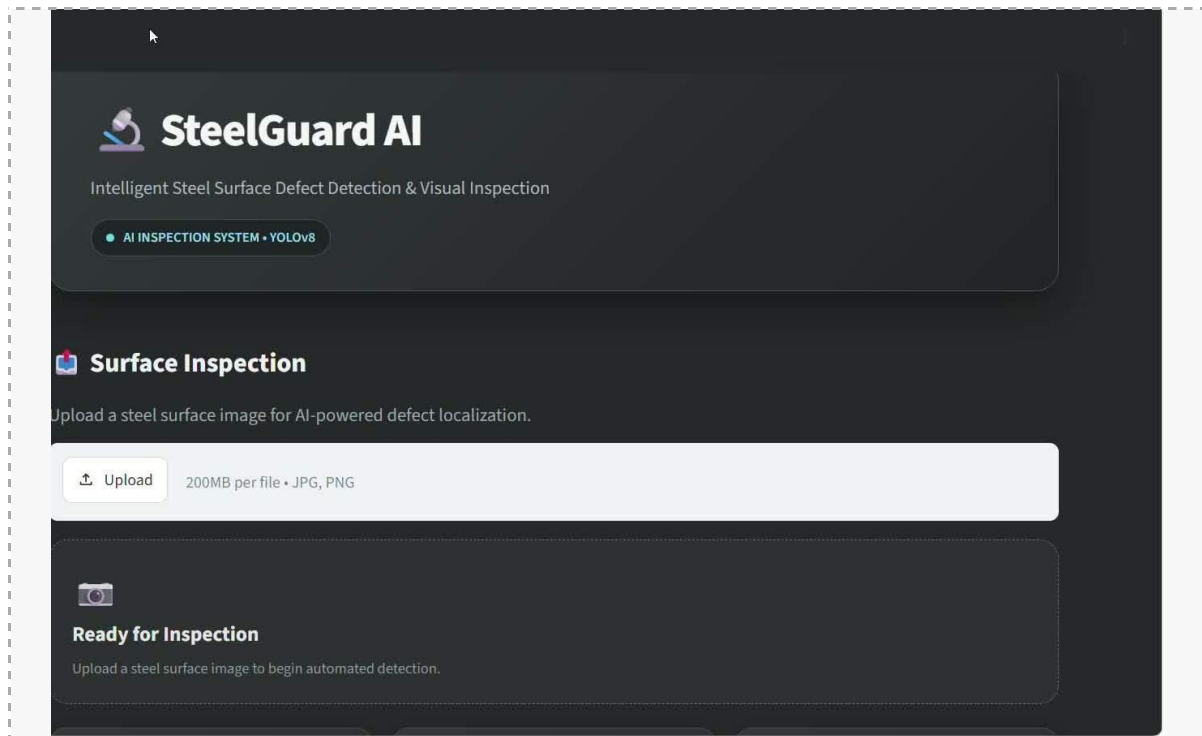


Figure 7: Sidebar — Inspection panel and defect classes

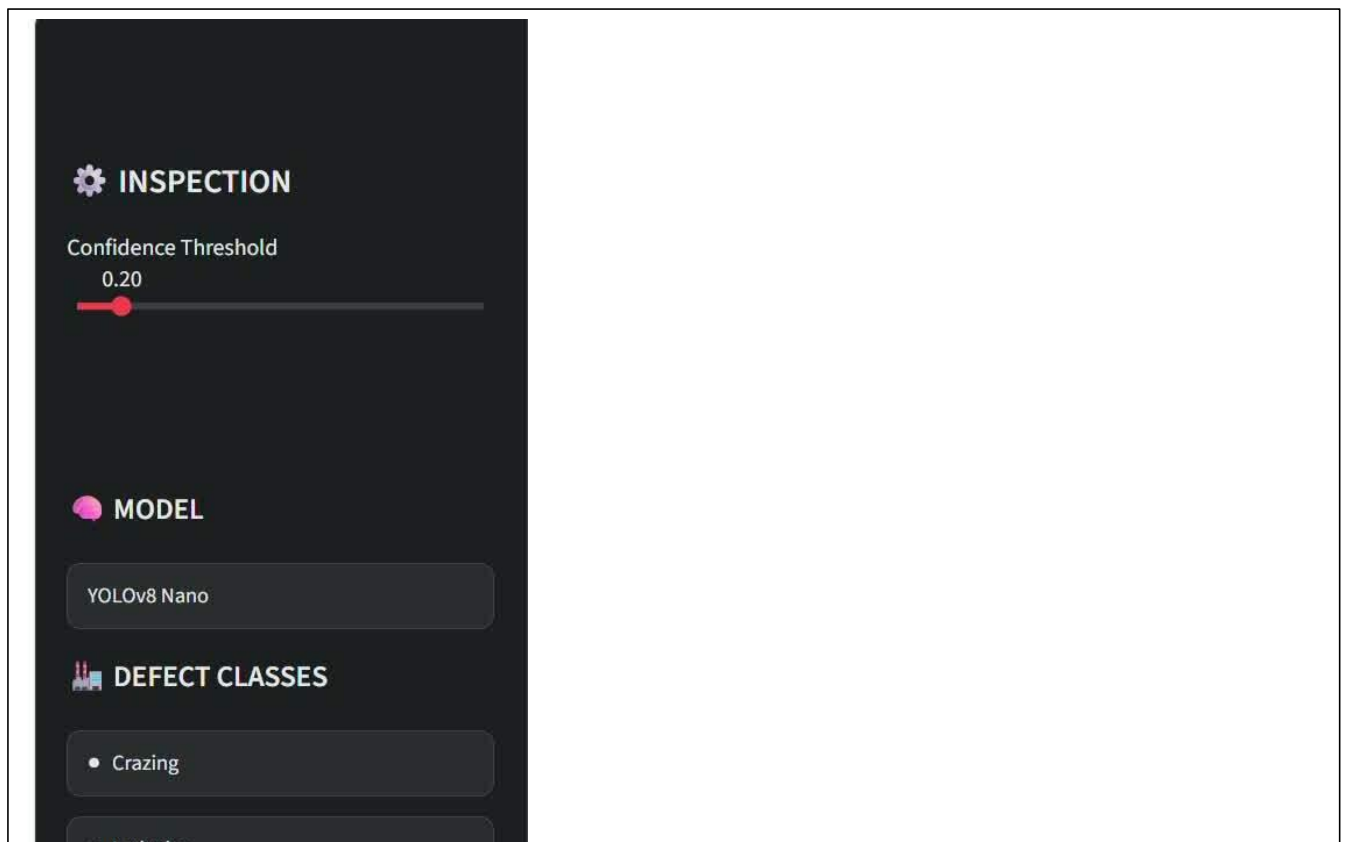


Figure 8: Image upload in progress

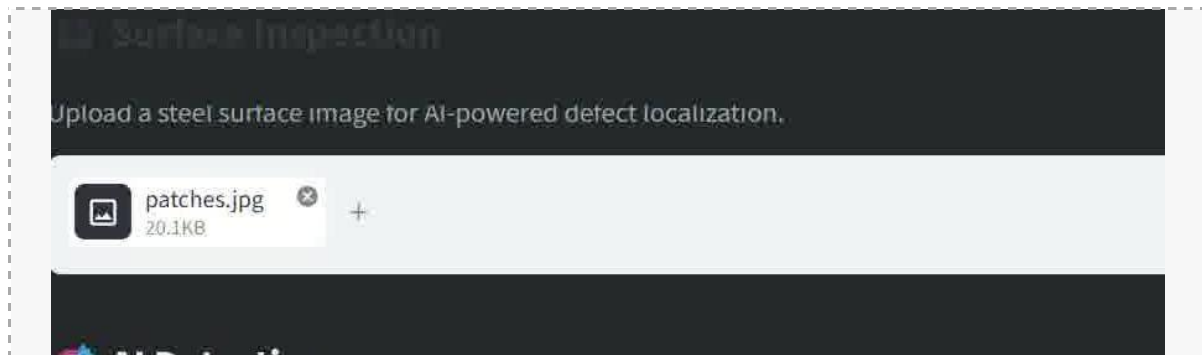


Figure 9: Original vs Detected image (AI Detection)

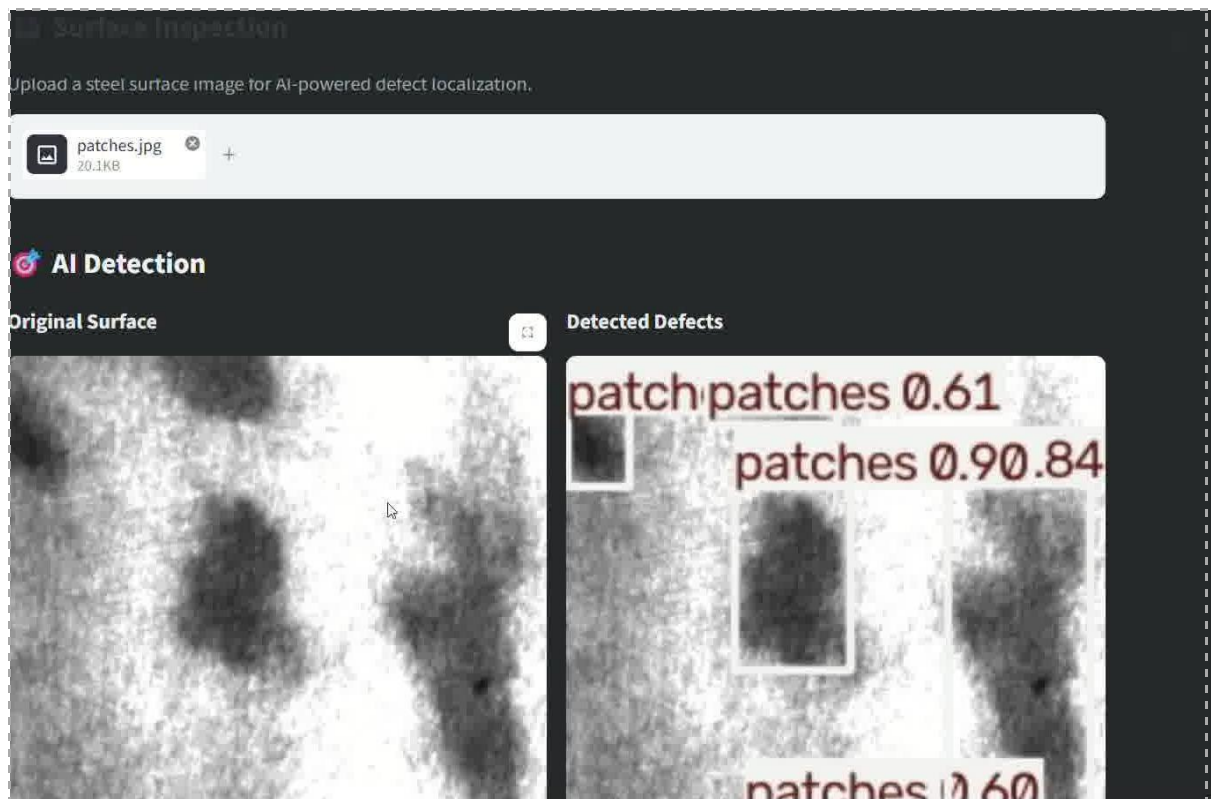


Figure 10: Detection Details table

Detection Details	
Defect	Confidence
patches	90.48%
patches	83.96%
patches	61.14%
patches	60.36%
patches	38.29%

Figure 11: Defect Distribution chart

